

```
-- SELECT
SELECT <колона 1>, <колона 2>, <агрегатна функция, напр. COUNT(*)>
FROM <име на основна таблица>

-- 1. СВЪРЗВАНЕ (Незадължително)
<LEFT/RIGHT/INNER> JOIN <име на друга таблица>
    ON <таблица1>.<колона> = <таблица2>.<колона>

-- 2. ФИЛТРИРАНЕ НА РЕДОВЕ (Незадължително)
WHERE <условие>
    -- Например: <колона> LIKE 'A%' (започва с A)

-- 3. ГРУПИРАНЕ (Незадължително)
GROUP BY <колона, по която обединяваме еднаквите записи>

-- 4. ФИЛТРИРАНЕ НА ГРУПИ (Незадължително)
HAVING <условие върху групите>
    -- Например: COUNT(*) > 1 (покажи само групите с повече от 1 запис)

-- 5. СОРТИРАНЕ (Незадължително)
ORDER BY <колона за сортиране> <ASC за възходящ или DESC за низходящ ред>

-- 6. ОГРАНИЧАВАНЕ НА БРОЯ (Незадължително)
LIMIT <брой редове, които искаш да видиш накрая>;
```

```
-- UPDATE
UPDATE <име на таблица>
SET <име на колона 1> = <нова стойност 1>,
    <име на колона 2> = <нова стойност 2>
WHERE <условие>; -- Например: id = 5
-- Ако забравиш WHERE, MySQL ще промени стойностите във ВСИЧКИ редове на таблицата!
```

```
-- INNER JOIN - Показва само редовете, които имат съвпадение и в двете таблици.
SELECT <име на таблица 1>.<колона 1>, <име на таблица 2>.<колона 2>
FROM <име на таблица 1>
INNER JOIN <име на таблица 2>
ON <име на таблица 1>.<колона за връзка> = <име на таблица 2>.<колона за връзка>;
```

```
-- LEFT JOIN - Показва всички редове от лявата таблица и съвпадащите от дясната таблица.
SELECT <лява таблица>.<колона 1>, <дясна таблица>.<колона 2>
FROM <лява таблица>
LEFT JOIN <дясна таблица> -- RIGHT JOIN <дясна таблица> прави обратното
ON <лява таблица>.<колона за връзка> = <дясна таблица>.<колона за връзка>;
```

```
-- FULL OUTER JOIN (Чрез UNION в MySQL)
SELECT <лява таблица>.<колона 1>, <дясна таблица>.<колона 2>
FROM <лява таблица>
LEFT JOIN <дясна таблица>
ON <лява таблица>.<колона за връзка> = <дясна таблица>.<колона за връзка>
```

UNION

```
SELECT <лява таблица>.<колона 1>, <дясна таблица>.<колона 2>
FROM <лява таблица>
RIGHT JOIN <дясна таблица>
ON <лява таблица>.<колона за връзка> = <дясна таблица>.<колона за връзка>;
```

Ето най-краткото и точно обобщение на четирите команди:

- **UPDATE** <ИМЕ НА ТАБЛИЦАТА> **SET** <ИМЕ НА КОЛОНАТА КОЯТО ПРОМЕНЯМЕ> = <НОВАТА СТОЙНОСТ>
- **WHERE** <ИМЕ НА КОЛОНАТА ЗА ТЪРСЕНЕ> = <СТАРАТА СТОЙНОСТ>
- **ON DELETE RESTRICT**: Забранява изтриването или промяната на родителя, ако към него има свързани зависими записи.
- **ON DELETE CASCADE**: Изтрива автоматично всички свързани "деца" (зависими записи), когато изтрием родителя.
- **ON UPDATE CASCADE**: Обновява автоматично ключовете на всички свързани "деца", когато се промени ключът на родителя.
- **ON DELETE SET NULL**: Премахва връзката при изтриване на родител, като запазва записите на "децата", но им задава стойност **NULL**.

```
START TRANSACTION;
```

```
-- Резервираме ред, преди да го променим
```

```
SELECT * FROM <име на таблица>
WHERE <име на колона> = <стойност>
FOR UPDATE;
```

```
UPDATE <име на таблица>
SET <име на колона за промяна> = <нова стойност>
WHERE <име на колона> = <стойност>;
SAVEPOINT <име на точка 1>;
```

```
UPDATE <име на таблица>
SET <име на колона за промяна> = <друга стойност>
WHERE <име на колона> = <друга стойност>;
ROLLBACK TO <име на точка 1>;
COMMIT;      /   ROLLBACK;
```

```
DELIMITER //
```

```
CREATE PROCEDURE <име на процедура> (
    IN coach_id INT,                -- IN - входен параметър
    OUT coach_name VARCHAR(100),    -- OUT - изходен параметър
    INOUT current_salary DOUBLE     -- INOUT - входно изходен параметър
)
```

```
BEGIN
```

```
    DECLARE bonus_percentage DOUBLE; -- Деклариране на локална променлива
    SET bonus_percentage = 0.10;
```

```
    -- 1. Намираме името по ID и го записваме в OUT параметъра
```

```
    SELECT name INTO coach_name
    FROM coaches
    WHERE id = coach_id;
```

```
    -- 2. Увеличаваме заплатата в таблицата със сумата от INOUT параметъра и 10% бонус от
    локалната променлива
```

```
    UPDATE coaches
    SET salary = salary + current_salary + (salary * bonus_percentage)
    WHERE id = input_id;
```

```
    -- 3. Записваме новата крайна заплата обратно в същия INOUT параметър
```

```
    SELECT salary INTO current_salary
    FROM coaches
    WHERE id = coach_id;
```

```
END //
```

```
DELIMITER;
```

```
SET @salary_to_add = 500; -- Задаваме сума за увеличение
CALL update_coach_data_advanced(1, @retrieved_name, @salary_to_add); -- Извикваме процедурата
SELECT @retrieved_name AS Coach, @salary_to_add AS FinalSalary; -- Виждаме резултата
```

-- Блок 1 изчислява и записва статуса в скрита локална променлива на базата на поредица от проверки.
-- Блок 2 е напълно независим и взема логическо решение дали въобще да изпълни SELECT заявката и да изведе резултат на екрана.

```
DELIMITER //
```

```
CREATE PROCEDURE evaluate_student(  
    IN p_grade INT,           -- Оценката на ученика (число)  
    IN p_show_result VARCHAR(10) -- Искаме ли да видим резултата ('Да' или 'Не')  
)  
BEGIN  
    -- 1. Декларираме локална променлива, в която ще запазим категорията  
    DECLARE v_category VARCHAR(20);  
  
    -- БЛОК 1: Навързана верига от условия (IF ... ELSEIF ... ELSE)  
    IF p_grade >= 5.50 THEN -- Ако е вярно  
        SET v_category = 'Отличник';  
    ELSEIF p_grade >= 5 THEN -- Ако е вярно  
        SET v_category = 'Много добър';  
    ELSE -- Ако не е вярно  
        SET v_category = 'Скъсан';  
    END IF; -- Този END IF затваря цялата верига по-горе!  
  
    -- БЛОК 2: Напълно независим втори IF  
    IF p_show_result = 'Да' THEN  
        -- Извличаме данните като таблица (SELECT)  
        SELECT p_grade AS 'Оценка', v_category AS 'Статус';  
    END IF; -- Този END IF затваря само втория блок!
```

```
END //
```

```
DELIMITER;
```

-- Тази процедура използва IF като функция директно в заявката за извличане на данни, без да се нуждае от BEGIN (за самата проверка) или END IF. Тя не управлява стъпките на самата програма, а просто минава през всеки ред на таблицата Students и динамично създава нова колона status, в която записва "Добър" или "Слаб" според оценката.

```
DELIMITER //
```

```
CREATE PROCEDURE get_student_statuses()  
BEGIN  
    -- Тук IF е просто функция вътре в заявката  
    SELECT name, grade, IF(grade >= 5, 'Добър', 'Слаб') AS status  
    FROM Students;  
END //
```

```
DELIMITER;
```

-- Simple CASE е когато търсим точно съвпадение на една колона с конкретни стойности

```
DELIMITER //
```

```
CREATE PROCEDURE get_user_status_bg(IN p_user_id INT)  
BEGIN  
    -- Използваме SELECT за да извлечем и преобразим данните  
    SELECT name,  
        CASE status -- CASE: <име на колоната>  
            WHEN 'active' THEN 'Активен потребител'  
            WHEN 'inactive' THEN 'Неактивен потребител'  
            ELSE 'Неизвестен статус'  
        END AS status_bg -- Задължително затваряме с END и даваме име на новата колона  
    FROM users  
    WHERE id = p_user_id; -- Заявката ще се изпълни САМО за подаденото ID  
END //
```

```
DELIMITER;
```

-- Search CASE е когато проверяваме цели логически условия, което ни позволява да използваме абсолютно всякакви оператори (>,<,BETWEEN...)

DELIMITER //

```
CREATE PROCEDURE update_all_coaches_bonuses() -- Вече няма входни параметри
BEGIN
    -- UPDATE заявката ще се изпълни за цялата таблица едновременно,
    -- защото няма WHERE клауза с ограничаващо / филтриращо условие след END
    UPDATE coaches
    SET bonus_amount = CASE category
        WHEN 'A' THEN 500
        WHEN 'B' THEN 300
        WHEN 'C' THEN 100
        ELSE 0
    END;
END //
```

DELIMITER;

Циклите (WHILE, REPEAT, LOOP) могат да се използват САМО в съхранени процедури, функции или тригери.

-- WHILE - условието се проверява в начало (преди изобщо да влезе в цикъла). Ако е грешно, цикълът няма да се изпълни

```
WHILE <условие> DO
    <заявки>;
END WHILE;
```

DELIMITER //

```
CREATE PROCEDURE update_top_coaches()
BEGIN
    DECLARE v_id INT DEFAULT 1; -- Започваме от треньор с ID 1

    WHILE v_id <= 5 DO -- Цикълът ще се завърти 5 пъти
        -- Всяка обиколка прави UPDATE на конкретно ID
        UPDATE coaches
        SET salary = salary + 100
        WHERE id = v_id; -- Ето тук броячът се среща с WHERE!

        SET v_id = v_id + 1; -- Увеличаваме ID-то за следващата стъпка
    END WHILE;
END //
```

DELIMITER ;

-- REPEAT цикъла ще се изпълни поне веднъж, дори условието за спиране да е грешно

```
REPEAT
    <заявки>;
UNTIL <условие>
END REPEAT;
```

```

DELIMITER //
CREATE PROCEDURE sum_numbers_repeat()
BEGIN
    DECLARE sum INT DEFAULT 0;
    DECLARE i INT DEFAULT 1;

    REPEAT
        SET sum = sum + i; -- Изпълняваме действието веднага (i е 1)
        SET i = i + 1;      -- Увеличаваме брояча (i става 2)
    UNTIL i > 10            -- Проверката на изхода: "i станало ли е по-голямо от 10?"
    END REPEAT;

    SELECT sum AS total_sum;
END//
DELIMITER ;
-- LOOP - безкраен цикъл

<име на цикъла>: LOOP
    <заявки>;
    IF <условие за прескачане> THEN      -- Прескачане!
        <заявки за изпълнение>;
        ITERATE <име на цикъла>;
    END IF;

    <заявки>;                            -- Основен код (НЯМА да се изпълни, ако горното условие е
    било вярно)
    IF <условие за изход> THEN
        LEAVE <име на цикъла>;          -- Спираме цикъла
    END IF;
END LOOP;
-----
DELIMITER //
CREATE PROCEDURE loop_example()
BEGIN
    DECLARE counter INT DEFAULT 1;
    DECLARE max_count INT DEFAULT 10;

    loop_start: LOOP -- Даваме име на цикъла "loop_start"

        -- 1. Демонстрация на ITERATE (Прескачане)
        IF counter = 5 THEN
            SET counter = counter + 1;
            ITERATE loop_start;
        END IF;

        -- 2. Демонстрация на LEAVE (Спиране)
        IF counter > max_count THEN [cite: 244]
            LEAVE loop_start;          -- Прекратява цикъла
        END IF; [cite: 245]

        SELECT counter; -- Показваме текущия брояч
        SET counter = counter + 1; -- Увеличаваме брояча

    END LOOP;
END//
DELIMITER ;

```

-- Временна таблица

```
CREATE TEMPORARY TABLE <име на таблица> (  
    <име на колона> INT PRIMARY KEY,  
    <имена на колона> VARCHAR(100),  
    <име на колона> DECIMAL(10, 2)  
);
```

-- Тригер

DELIMITER //

```
CREATE TRIGGER <име на тригера>  
<Време - BEFORE / AFTER> <Събитие - INSERT / UPDATE / DELETE>  
ON <име на таблицата>  
FOR EACH ROW  
BEGIN  
  
    -- <Заявки и логика> (проверки, сметки, записи в други таблици)  
  
    -- Пример за проверка на входящи данни с NEW:  
    IF NEW.<име на колона> <условие> THEN  
        SET NEW.<име на колона> = <стойност>;  
    END IF;  
  
    -- Пример за взимане на стари данни с OLD:  
    INSERT INTO <архивна таблица> (<колона 1>) VALUES (OLD.<име на колона>);  
  
END //
```

DELIMITER ;

-- 1. Показва абсолютно всички тригери в текущата (активно избраната) база данни
SHOW TRIGGERS;

-- 2. Показва всички тригери, но от друга, конкретно посочена база данни
SHOW TRIGGERS FROM <име на база данни>;

-- 3. Показва тригери (в текущата база), чиито ИМЕНА съвпадат с търсения шаблон (напр. 'before_%')
SHOW TRIGGERS LIKE '<шаблон за търсене>;

-- 4. Търси тригери в конкретна база данни по шаблон за ИМЕ
SHOW TRIGGERS FROM <име на база данни> LIKE '<шаблон за търсене>;

-- 5. Показва тригери по специфично УСЛОВИЕ (Най-често: за да видим тригерите на конкретна ТАБЛИЦА)
SHOW TRIGGERS WHERE `Table` = '<име на таблицата>;

```

-- Курсор
DELIMITER//
CREATE PROCEDURE <име на процедура> ()
BEGIN
    DECLARE <име на променлива> <тип>;
    DECLARE <име на курсора> CURSOR FOR
        SELECT <колона 1>, <колона 2>
        FROM <име на таблица>;

    DECLARE CONTINUE HANDLER FOR NOT FOUND <условие>    -- например: SET done = 1;
    OPEN <име на курсора>;

    <име на цикъла>: LOOP
        FETCH <име на курсора> INTO <променлива 1>, <променлива 2>;
        IF done THEN
            LEAVE <име на цикъла>;
        END IF;
    END LOOP;

CLOSE <име на курсора>;
=====
DELIMITER //
CREATE PROCEDURE process_all_employees()
BEGIN
    -- 1. ДЕКЛАРИРАНЕ НА ПРОМЕНЛИВИ (Винаги първи по закон)
    DECLARE done INT DEFAULT 0;                -- Флаг за край на редовете
    DECLARE v_name VARCHAR(100);                -- Променлива за името
    DECLARE v_salary DECIMAL(10,2);            -- Променлива за заплатата
    DECLARE v_bonus DECIMAL(10,2);            -- Променлива за новия бонус

    -- 2. ДЕКЛАРИРАНЕ НА КУРСОР (Винаги след променливите) кои колони от коя таблица да "гледа"
    DECLARE emp_cursor CURSOR FOR
        SELECT name, salary FROM employees;

    -- 3. ДЕКЛАРИРАНЕ НА ОБРАБОТВАНЕ НА ГРЕШКИ (HANDLER)- "слуша" за края на редовете
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    OPEN emp_cursor;                -- 4. ОТВАРЯНЕ НА КУРСОР (Извън цикъла)

    read_loop: LOOP                -- 5. БЕЗКРАЕН ЦИКЪЛ (LOOP)

        -- 6. ИЗВЛИЧАНЕ НА ДАННИТЕ (FETCH) - "Наливаме" данните от текущия ред на курсор в нашите променливи
        FETCH emp_cursor INTO v_name, v_salary;

        -- 7. ПРОВЕРКА ЗА КРАЙ (Използваме флага от хандлера)
        IF done THEN
            LEAVE read_loop; -- Ако редовете са свършили, бягаме от цикъла
        END IF;

        -- 8. РАБОТА/ЛОГИКА (Пример: бонус 10% от заплатата)
        SET v_bonus = v_salary * 0.10;

        -- Обновяваме реалната таблица с новите данни
        UPDATE employees
        SET bonus_amount = v_bonus
        WHERE name = v_name;

        -- Показваме междинен резултат за проверка
        SELECT v_name AS 'Служител', v_bonus AS 'Изчислен Бонус';
    END LOOP; -- Край на цикъла

    -- 9. ЗАТВАРЯНЕ НА КУРСОР (Освобождаваме паметта)
    CLOSE emp_cursor; --
END //
DELIMITER ;

```